Ride Booking Big Data Project – Hive Data Cleaning Workflow

1. Create HDFS Directory and Upload Dataset

Create HDFS Directory

```
# Create project directory in HDFS

hdfs dfs -mkdir /ride
```

Upload Dataset to HDFS

```
# Upload CSV file into HDFS

hdfs dfs -put Bookings.csv /ride/
```

Verify Uploaded File

```
# List files in HDFS directory

hdfs dfs -ls /ride
```

Check Dataset Header

```
# Display first row

hdfs dfs -cat /ride/Bookings.csv | head -1
```

Display Column Names

```
# Display each column on a separate line

hdfs dfs -cat /ride/Bookings.csv | head -1 | tr ',' '\n'
```

---

2. Create Hive Raw Table

Purpose

Create a Hive table to store the original booking dataset before performing data cleaning operations.

```
CREATE TABLE bookings_raw (

Booking_Date STRING,
Booking_Time STRING,
Booking_ID STRING,
Booking_Status STRING,
Customer_ID STRING,
Vehicle_Type STRING,
Pickup_Location STRING,
Drop_Location STRING,

V_TAT STRING,
C_TAT STRING,

Cancelled_Rides_by_Customer STRING,
Reason_for_cancelling_by_Customer STRING,

Cancelled_Rides_by_Driver STRING,
Driver_Cancellation_Reason STRING,

Incomplete_Rides STRING,
Incomplete_Rides_Reason STRING,

Booking_Value STRING,
Payment_Method STRING,
Ride_Distance STRING,
Driver_Ratings STRING,
Customer_Rating STRING

)

ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE

TBLPROPERTIES (
"skip.header.line.count"="1"
);
```

3. Load Dataset into Hive

Purpose

Load the booking dataset stored in HDFS into the Hive raw table.

```
LOAD DATA INPATH '/ride/Bookings.csv'
INTO TABLE bookings_raw;
```

---

4. Verify Raw Data

Check Table Structure

```
DESCRIBE bookings_raw;
```

Check Total Records

```
SELECT COUNT(*)
FROM bookings_raw;
```

Preview Dataset

```
SELECT *
FROM bookings_raw
LIMIT 10;
```

---

5. Data Cleaning and Transformation

Purpose

The following cleaning operations are performed:

- Remove duplicate records
- Replace invalid "null" strings with actual NULL values
- Convert numeric fields to DOUBLE data type

- Standardize data format
- Create a cleaned dataset for further analytics

Create Cleaned Table

```
DROP TABLE IF EXISTS bookings_clean;

CREATE TABLE bookings_clean AS

SELECT DISTINCT

Booking_Date,
Booking_Time,
Booking_ID,
Booking_Status,
Customer_ID,
Vehicle_Type,
Pickup_Location,
Drop_Location,

CAST(NULLIF(V_TAT,'null') AS DOUBLE)
AS V_TAT,

CAST(NULLIF(C_TAT,'null') AS DOUBLE)
AS C_TAT,

NULLIF(Cancelled_Rides_by_Customer,'null')
AS Cancelled_Rides_by_Customer,

NULLIF(Reason_for_cancelling_by_Customer,'null')
AS Reason_for_cancelling_by_Customer,

NULLIF(Cancelled_Rides_by_Driver,'null')
AS Cancelled_Rides_by_Driver,

NULLIF(Driver_Cancellation_Reason,'null')
AS Driver_Cancellation_Reason,

NULLIF(Incomplete_Rides,'null')
AS Incomplete_Rides,

NULLIF(Incomplete_Rides_Reason,'null')
AS Incomplete_Rides_Reason,

CAST(NULLIF(Booking_Value,'null') AS DOUBLE)
```

```
AS Booking_Value,

NULLIF(Payment_Method,'null')
AS Payment_Method,

CAST(NULLIF(Ride_Distance,'null') AS DOUBLE)
AS Ride_Distance,

CAST(NULLIF(Driver_Ratings,'null') AS DOUBLE)
AS Driver_Ratings,

CAST(NULLIF(Customer_Rating,'null') AS DOUBLE)
AS Customer_Rating

FROM bookings_raw;
```

---

6. Validate Cleaned Dataset

Verify Table Creation

```
SHOW TABLES;
```

Check Table Schema

```
DESCRIBE bookings_clean;
```

Verify Record Count

```
SELECT COUNT(*)
FROM bookings_clean;
```

Preview Cleaned Records

```
SELECT *
FROM bookings_clean
LIMIT 20;
```

---

7. Check Missing Values

Purpose

Evaluate data completeness after cleaning.

```
SELECT

COUNT(*) AS Total_Rows,

COUNT(V_TAT) AS Valid_VTAT,

COUNT(C_TAT) AS Valid_CTAT,

COUNT(Ride_Distance) AS Valid_Ride_Distance,

COUNT(Driver_Ratings) AS Valid_Driver_Ratings,

COUNT(Customer_Rating) AS Valid_Customer_Ratings

FROM bookings_clean;
```

---

8. Check Duplicate Records

Purpose

Ensure that duplicate Booking IDs have been removed.

```
SELECT
Booking_ID,
COUNT(*) AS Duplicate_Count

FROM bookings_clean

GROUP BY Booking_ID

HAVING COUNT(*) > 1;
```

---

9. Export Cleaned Dataset to HDFS

Purpose

Store the cleaned dataset in HDFS for downstream analytics and Spark processing.

```
INSERT OVERWRITE DIRECTORY '/ride_output'

ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','

SELECT *

FROM bookings_clean;
```

---

10. Download Cleaned Dataset from HDFS

```
# Merge output files into a single CSV file

hdfs dfs -getmerge /ride_output ~/Bookings_Cleaned_Hive.csv
```

Verify Exported File

```
head ~/Bookings_Cleaned_Hive.csv
```

---

11. Store Final Dataset in HDFS

Create Final Storage Directory

```
hdfs dfs -mkdir -p /ride_final
```

Upload Final Cleaned Dataset

```
hdfs dfs -put -f ~/Bookings_Cleaned_Hive.csv /ride_final/
```

Verify Uploaded Dataset

```
hdfs dfs -ls /ride_final
```

---

12. Final Validation

```
hdfs dfs -cat /ride_final/Bookings_Cleaned_Hive.csv | head
```

Expected Result:

- Cleaned dataset successfully exported
- Duplicate records removed
- Invalid NULL values handled
- Numeric attributes converted into appropriate data types
- Dataset ready for Spark SQL analytics and Machine Learning processing

---

Summary of Hive Data Cleaning Tasks

✔ Dataset uploaded into HDFS

✔ Raw Hive table created

✔ Data loaded into Hive

✔ Duplicate records removed

✔ Missing values handled

✔ Data type conversion completed

✔ Cleaned table generated

✔ Dataset validated

✔ Exported back to HDFS

✔ Prepared for Spark Analytics